1. Introduction: Shooter vs Target Animation using Turbo C++ Graphics

In the world of computer programming, the ability to create engaging and interactive graphics is a critical skill, particularly for those interested in game development, simulations, or user-interface design. This micro project titled “Shooter vs Target Animation” explores the foundations of graphical programming in the C language, using the graphics.h library available in Turbo C++. By employing simple shapes, lines, and colors, the project simulates a shooting game where a police officer (referred to as the shooter) fires a bullet at a target on the opposite side of the screen.

The primary goal of this project is to introduce the basics of 2D animation in C, focusing on the use of primitive graphical functions such as drawing circles, rectangles, and lines, as well as animating these shapes to create a coherent motion. In this particular animation, the shooter’s gun fires a bullet toward the target, and the bullet moves across the screen in real-time. This process is repeated multiple times to emphasize the concept of looped animations. Additionally, the graphical elements of the shooter and target are continuously redrawn during each iteration, demonstrating how to manipulate coordinates and refresh graphics on the screen to create smooth movement.

A secondary goal of this project is to illustrate how graphical programming can be a stepping stone toward more advanced topics like 3D animation, game physics, and interactive media. The project provides a practical example of how simple concepts like object positioning, movement, and screen clearing can be applied to develop interactive graphical simulations. Moreover, by learning to utilize the graphics.h library, students and beginner programmers can gain an understanding of how pixels are controlled and manipulated on the screen. This project also serves as an excellent foundation for learning more complex features of game development and other graphical user interface (GUI) applications.

From a technical standpoint, the project requires the Turbo C++ environment to run. Turbo C++ is one of the most widely used development environments for C and C++ programming, especially for beginners. The graphics.h library, which comes with Turbo C++, provides a convenient and easy-to-understand set of functions for drawing shapes, filling colors, and performing animations, making it an ideal tool for this project. Although newer technologies and compilers are available today, Turbo C++ remains a popular choice for educational purposes due to its simplicity and ease of setup.

In conclusion, this micro project is an excellent starting point for those who wish to explore the fundamentals of computer graphics. By simulating a basic shooter-target animation, students can gain practical experience in manipulating graphical elements on the screen, understanding the principles of animation, and working with Turbo C++'s graphical capabilities. This knowledge can later be extended to more complex programs and games, providing a solid foundation for future work in graphics programming.

2. Requirement Specification for Project

- **Software**

The software requirements for this project ensure that the shooter vs target animation program runs effectively in a development environment that supports graphical operations using the graphics.h library in Turbo C++. Below are the key software tools needed to complete the project:

---

1. Turbo C++ Compiler (version 3.0 or compatible)

Turbo C++ is an integrated development environment (IDE) and compiler for C/C++ programming. It is one of the most widely used tools for developing applications in the C language, particularly in educational contexts due to its simplicity and ease of use. The graphics.h library, which comes pre-installed with Turbo C++, is used for rendering graphical shapes and animations.

In this project, Turbo C++ is essential because:

- It provides built-in functions to create and manipulate graphical elements like circles, rectangles, lines, and filling colors, which are the core of the shooter and target animation.
- It has a simple and lightweight interface, making it accessible for beginner-level projects.
- Turbo C++ enables the use of real-mode DOS graphics, which simplifies the execution of animations in small-scale projects.

2. Turbo C++ graphics.h Library

The graphics.h library is a crucial part of this project, as it provides the necessary functions for:

- Drawing basic shapes like circles, rectangles, and lines to form the shooter, target, and bullet.
- Filling areas of the screen with specific colors.
- Managing the graphical display, such as clearing the screen (cleardevice()), detecting errors (graphresult()), and closing the graphics window (closegraph()).

Some of the key functions from this library that are used in the project include:

- initgraph(): Initializes the graphics mode.
- circle(), rectangle(), line(): Used to draw shapes for the shooter, target, and bullet.
- floodfill(): Fills closed shapes with a specific color.
- setcolor(): Sets the color for drawing shapes and outlines.
- delay(): Pauses the animation temporarily, controlling the speed of the bullet movement.

The simplicity and ease of use of the graphics.h library make it an ideal tool for beginners learning graphical programming in C.

3. Windows Operating System

Although newer IDEs and compilers exist for C programming, Turbo C++ is often used on older operating systems like Windows XP, Windows 7, or even Windows 10/11. Windows OS provides compatibility for running DOS-based applications, which is important because Turbo C++ and its graphics mode rely on DOS functionality.

For running the project effectively, the following is required:

- **Windows XP/7/10/11**: A Windows-based operating system is recommended for Turbo C++ because of its DOS-based architecture. Even though Turbo C++ is outdated, it is still widely used in educational settings where these older versions of the OS are supported.

4. Turbo C++ BGI Driver Path Configuration

To enable graphical programming in Turbo C++, it is important to configure the path to the Borland Graphics Interface (BGI) driver, which is necessary for initiating graphics mode. This driver is used by initgraph() to switch the system into graphics mode and start rendering graphics.

For this project, the BGI driver is configured in the code with the path:

```
initgraph(&gdriver, &gmode, "C:/TURBOC3/BGI");
```

This line of code ensures that the correct graphics driver is loaded before the shooter-target animation is displayed on the screen.

- **System:**

Edition : Windows 11 Pro
Version :23H2
Installed on :25/9/2024
OS build :22631.4317
ExperienceWindows Feature Experience Pack 1000.22700.1041.0

- **HARDWARE:**

Device name : SAHIL

Processor :12th Gen Intel(R) Core (TM) i5-12400 2.50 GHz

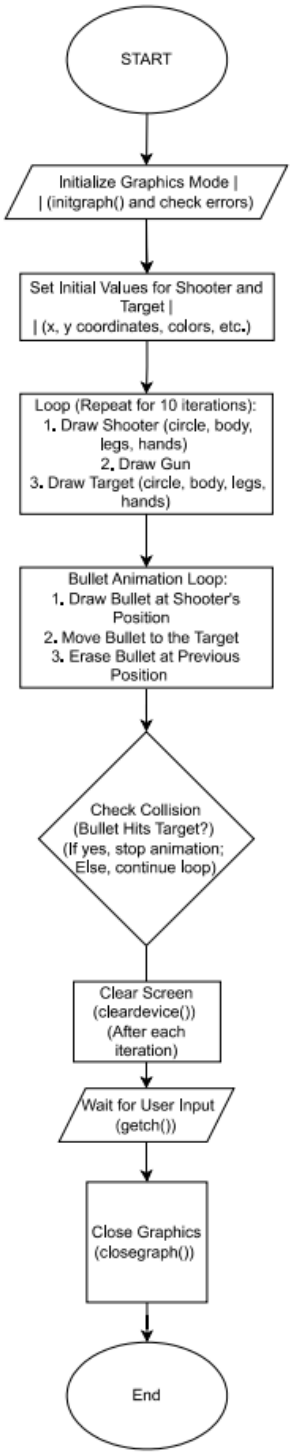
Installed RAM : 16.0 GB (15.7 GB usable) (DDR4)

Device ID :3A25A360-23C0-4631-9527-54163794A930

Product ID :00331-10000-00001-AA686

System type :64-bit operating system, x64-based processor

3. FLOWCHART:


START
Initialize Graphics Mode |
| (initgraph() and check errors)
Set Initial Values for Shooter and Target |
| (x, y coordinates, colors, etc.)
Loop (Repeat for 10 iterations):
1. Draw Shooter (circle, body, legs, hands)
2. Draw Gun
3. Draw Target (circle, body, legs, hands)
Bullet Animation Loop:
1. Draw Bullet at Shooter's Position
2. Move Bullet to the Target
3. Erase Bullet at Previous Position
Check Collision
(Bullet Hits Target?)
(If yes, stop animation;
Else, continue loop)
Clear Screen
(cleardevice())
(After each iteration)
Wait for User Input
(getch())
Close Graphics
(closegraph())
End

4. Algorithm:

1. Start the program.
2. Initialize graphics mode:
    - Call initgraph() function to start the graphics mode.
    - Check if there is any error in the graphics initialization using graphresult ().
    - If an error occurs, display the error message and terminate the program.
3. Set initial coordinates for the shooter and the target:
    - Assign initial values for the shooter's position (x, y) and the target's position (target coordinates based on the screen width).
    - Set colors for the shooter, target, and bullet.
4. Start the main animation loop (Repeat 10 times):
    - Step 1: Draw the shooter.
        - Use the circle () function to draw the shooter's head.
        - Use the line () function to draw the shooter's body, hands, legs, and gun.
        - Fill the shapes with appropriate colors using floodfill ().
    - Step 2: Draw the target.
        - Use circle () for the head of the target.
        - Use line () for the body, hands, and legs of the target.
        - Fill the shapes with appropriate colors using floodfill ().
5. Start bullet motion loop:
    - Set the initial position for the bullet (near the shooter's gun).
    - In a loop, move the bullet from the shooter's position towards the target:
        - Draw the bullet using rectangle () at its current position.
        - Use floodfill () to fill the bullet shape with color.
        - Delay the animation using delay () to slow down the bullet movement.
        - Erase the bullet from the previous position by drawing a black rectangle over it (background color).
        - Increment the bullet's position (move it right on the screen).
6. Check collision:
    - Check if the bullet reaches the target by comparing the bullet's x-coordinate with the target's x-coordinate.
    - If the bullet hits the target:
        - Stop the bullet movement and break the loop.
        - Display the final bullet position on the target.
7. Clear the screen after each iteration using cleardevice () to prepare for the next animation loop.
8. Repeat steps 4 to 7 until all 10 iterations of the main loop are completed.
9. Wait for user input using getch () to keep the final screen displayed until the user presses a key.
10. Close the graphics mode using closegraph () to deallocate memory and exit graphics mode.
11. End the program.

5. Main code :

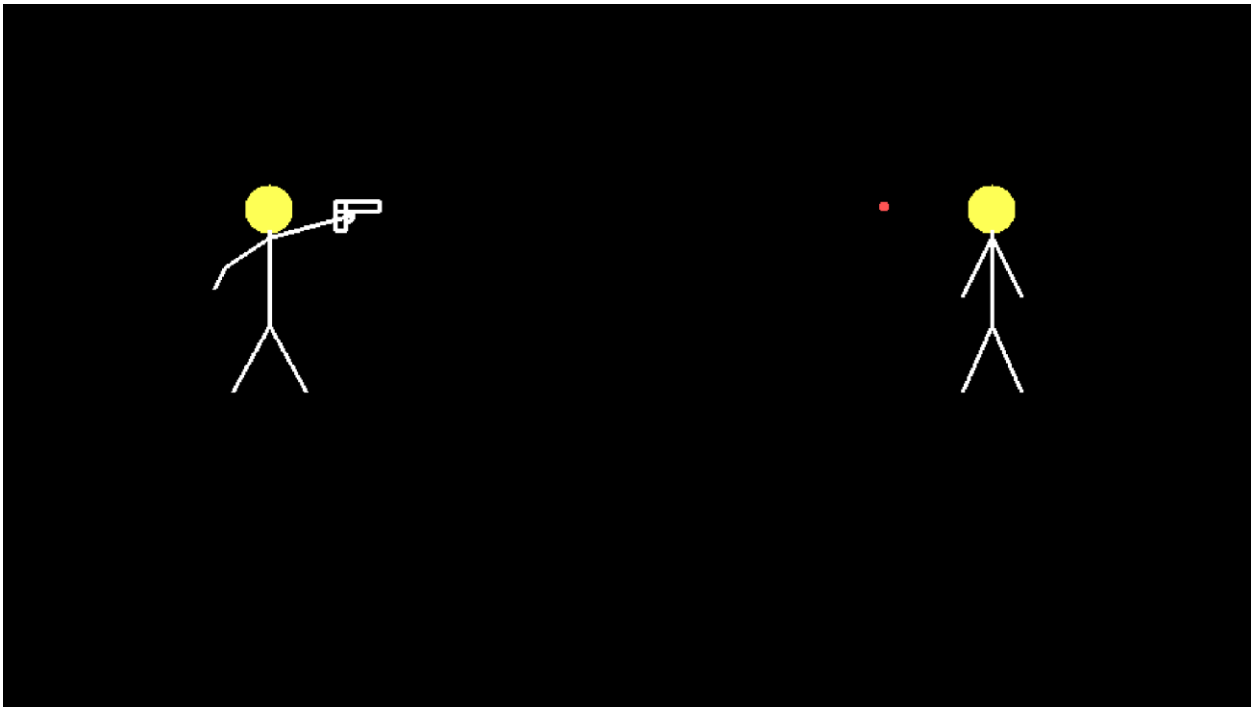
```
#include <stdio.h>

#include <conio.h>

#include <graphics.h>

#include <dos.h>

int main() {

    /* request auto detection */

    int gdriver = DETECT, gmode, err;

    int i, j, x, y, top[2], bot[2];

    /* initialize graphic mode */

    initgraph(&gdriver, &gmode, "C:/TURBOC3/BGI");

    err = graphresult();

    if (err != grOk) {

        /* error occurred */

        printf("Graphics Error: %s", grapherrormsg(err));

        return 0;

    }

    x = 75;

    y = getmaxy() / 2 - 100;


    for (i = 0; i < 10; i++) {

        /* draw the police (shooter) */

        setcolor(YELLOW);

        setlinestyle(SOLID_LINE, 1, 3);

        setfillstyle(SOLID_FILL, YELLOW);

        /* drawing head of shooter */
```

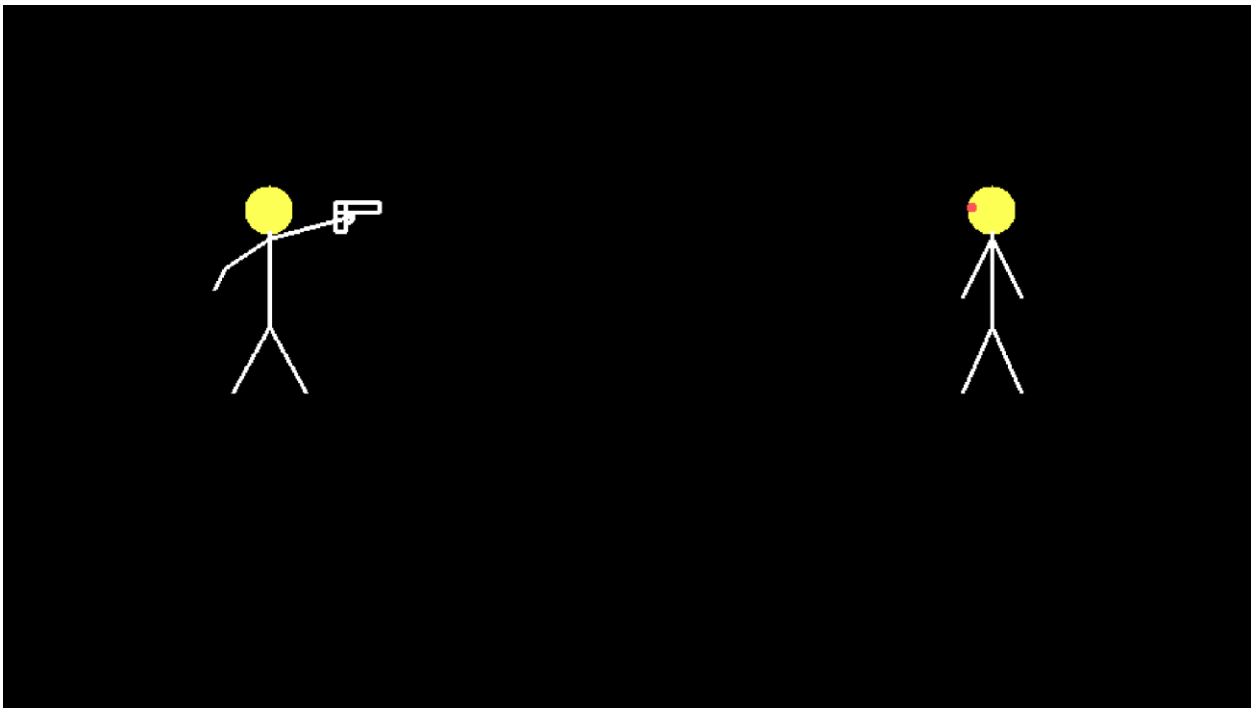
```
circle(x, y, 15);
floodfill(x, y, YELLOW);
/* drawing body */
setcolor(WHITE);
line(x, y + 15, x, y + 80);
/* leg design */
line(x, y + 80, x - 25, y + 125);
line(x, y + 80, x + 25, y + 125);
/* hand design */
line(x, y + 20, x - 30, y + 40);
line(x - 30, y + 40, x - 38, y + 55);
line(x, y + 20, x + 55, y + 5);
/* gun design */
rectangle(x + 45, y - 5, x + 52, y + 15);
rectangle(x + 45, y - 5, x + 75, y + 2);
arc(x + 52, y + 3, 270, 360, 6);
delay(300);
/* target */
setcolor(YELLOW);
setfillstyle(SOLID_FILL, YELLOW);
/* head of target */
circle(getmaxx() - 70, y, 15);
floodfill(getmaxx() - 70, y, YELLOW);
setcolor(WHITE);
/* body of the target */
line(getmaxx() - 70, y + 15, getmaxx() - 70, y + 80);
```

```
/* hand and legs target */
line(getmaxx() - 70, y + 20, getmaxx() - 50, y + 60);
line(getmaxx() - 70, y + 20, getmaxx() - 90, y + 60);
line(getmaxx() - 70, y + 80, getmaxx() - 90, y + 125);
line(getmaxx() - 70, y + 80, getmaxx() - 50, y + 125);
top[0] = x + 78, top[1] = y - 3;
bot[0] = x + 82, bot[1] = y;
/* motion of bullet */
for (j = top[0]; j < getmaxx() - 80; j = j + 10) {
    /* draws bullet at the current given position */
    setcolor(LIGHTRED);
    setfillstyle(SOLID_FILL, LIGHTRED);
    rectangle(top[0], top[1], bot[0], bot[1]);
    floodfill(top[0] + 1, top[1] + 1, LIGHTRED);
    delay(10);
    /* erase bullet at the previous position */
    setcolor(BLACK);
    setfillstyle(SOLID_FILL, BLACK);
    rectangle(top[0], top[1], bot[0], bot[1]);
    floodfill(top[0] + 1, top[1] + 1, BLACK);
    top[0] = top[0] + 10;
    bot[0] = bot[0] + 10;
}
/* bullet at the top of target */
setcolor(LIGHTRED);
setfillstyle(SOLID_FILL, LIGHTRED);
```

```
        rectangle(top[0] - 10, top[1], bot[0] - 10, bot[1]);
        floodfill(top[0] - 9, top[1] + 1, LIGHTRED);
        sleep(1);
        cleardevice();
    }
    getch();
    /* deallocate memory allocated for graphic screen */
    closegraph();
    return 0;
}
}
```

6. OUTPUT:





7. CONCLUSION:

The "Shooter vs Target" animation project, implemented using Turbo C++ and the graphics.h library, showcases the application of basic computer graphics principles in C programming. This project provides an excellent learning opportunity by demonstrating the integration of different graphical elements and their interactions in a simulated environment.

In this project, we were able to simulate a shooting scenario where a shooter character fires a bullet at a target. Both the shooter and the target were created using primitive shapes such as circles, lines, and rectangles. These shapes were combined to represent the figures, while motion was simulated using loops, delays, and coordinate transformations to animate the bullet's trajectory from the shooter toward the target.

Key concepts utilized in this project include graphical object creation, where functions such as circle(), line(), rectangle(), and floodfill() were employed to draw the shooter, the gun, and the target. The motion of the bullet was achieved by incrementally updating its position in the loop and rendering it on the screen at each iteration. The use of cleardevice() and delay() helped in creating smooth transitions and realistic animation effects, giving the impression of a continuous motion sequence.

This project highlights the power of graphical programming in C, illustrating how visual elements can be manipulated to create animations. By breaking down complex tasks into smaller steps such as shape rendering and motion handling, we were able to simulate realistic shooting and hitting effects. It also emphasizes the importance of timing in animations, where proper delays and screen clearing play a crucial role in maintaining fluidity.

Furthermore, this project serves as a stepping stone towards more advanced graphics programming concepts such as game development, physics simulations, and real-time rendering. It encourages further exploration of graphical libraries and more sophisticated animations, laying a foundation for future projects involving object interaction, collision detection, and enhanced visual effects.

In conclusion, the "Shooter vs Target" animation project demonstrates how basic shapes and motion can be combined to create dynamic simulations. Through this exercise, the fundamental building blocks of computer graphics are brought to life, offering both a fun and educational experience in the field of programming and animation.